

Cost-Based Query Optimization for Quantum Computation

Immanuel Trummer

Cornell University
Ithaca, New York, USA
itrummer@cornell.edu

Abstract

Prior work explores the use of quantum computing to solve variants of query optimization. Instead, I propose using query optimization techniques to make quantum computation more efficient.

CCS Concepts

• **Hardware** → **Quantum computation**; • **Information systems** → **Query optimization**.

Keywords

query optimization, quantum computation, multi-objective

ACM Reference Format:

Immanuel Trummer. 2025. Cost-Based Query Optimization for Quantum Computation. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '25)*, June 22–27, 2025, Berlin, Germany. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3736393.3736690>

Introduction. Solving a problem using current, Noisy Intermediate-Scale (NISQ)-era quantum computers is typically a multi-step process. Due to the limited number of qubits, problem instances of realistic sizes have to be decomposed into smaller problems that can be solved on the quantum hardware. The problem format supported by current quantum hardware is often restricted (e.g., in terms of the problem type and connection structure between problem variables), requiring a problem transformation.

For each of those steps, various different implementations are available, associated with different tradeoffs between computation overheads and result quality. For instance, for the NP-hard problem of finding optimal minor embeddings [3], a transformation required to solve instances of quadratic unconstrained binary optimization, cheap heuristics [2] as well as exhaustive algorithms [1] are available. At the same time, the increasing availability of a variety of quantum devices in the Cloud makes it easy to switch between different quantum devices on a case-by-case basis. Furthermore, many computation steps feature tunable parameters (e.g., the number of runs on a quantum annealer to solve a specific sub-problem), allowing users to trade between computation cost and result quality. Finally, having access to multiple classical and quantum devices in the Cloud enables choices in terms of which steps are performed in parallel.

Altogether, this creates a large search space for implementing quantum computation. To remove the burden on users to choose an optimal setup, this abstract proposes approaches inspired by cost-based query optimization in database systems [6].

Plan Metrics. Multiple metrics are relevant to judge quantum computation plans (QCPs). A first relevant metric is run time. Run time depends on the number of steps performed, on the implementation used for each step and associated parameter settings, the amount of input data for each step, and parallelization. A second relevant metric is computation fees. Typically, fees for computation on actual quantum devices dominate total processing fees. Cloud providers of quantum devices often charge an amount that depends on the hardware and the computation time used. For fixed hardware, costs are proportional to computation time, except that parallelization does not impact costs. Finally, the quality of results is relevant to users and depends on properties of the QCP. Specifically, it depends on the properties of the hardware (e.g., the degree of noise impacting quantum computation) and of the implementation chosen for each step. We assume that result quality is measured by a domain-specific cost metric (e.g., plan execution costs when using quantum computation for query optimization [5]), enabling us to express quality as the ratio between the cost achieved by a QCP, relative to the theoretical optimum for the input problem.

Cost Models. As in classical query optimization, given a QCP, the first goal is to estimate the sizes of intermediate results. For instance, this entails estimating the number of sub-problems that result from applying specific decomposition methods. It also entails estimating the size of problem instances (e.g., minor embedding) that must be solved for specific steps and sub-problems. Next, we must estimate the run time for each processing step, based on the input size. From that, we can derive the processing time of the entire QCP. Similarly, we can estimate monetary costs by taking into account time estimates and the fees per computation time unit. Finally, we can estimate the quality of the QCP result, aggregating quality loss over all processing steps such as decomposition and noisy computation of solutions to sub-problems.

Analytical formulas that reliably link cost and quality estimates of QCP processing steps to input and hardware properties are not available. However, prior work on learned query optimization shows that query optimizers can learn complex cost functions by profiling sample plans over time [4].

Problem Statement. Given an optimization problem o , denote by $\mathcal{P}(o)$ the space of QCPs and by $Cost(p) \in \mathbb{R}^3$ for $p \in \mathcal{P}(o)$ the three-dimensional cost vector (time, fees, and result cost overheads) of the plan. Given user-defined constraints $c \in \mathbb{R}^3$ and weighted preferences $w \in \mathbb{R}^3$, the goal of QCP optimization is to find the plan $p \in \mathcal{P}(o)$ minimizing $w \cdot Cost(p)$, the dot product between cost and weights, while satisfying the constraint $Cost(p) \leq c$ (i.e., the cost is below the constraint value for each cost metric).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

Q-Data '25, Berlin, Germany

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1944-8/25/06

<https://doi.org/10.1145/3736393.3736690>

References

- [1] David E. Bernal, Kyle E.C. Booth, Raouf Dridi, Hedayat Alghassi, Sridhar Tayur, and Davide Venturelli. 2020. Integer Programming Techniques for Minor-Embedding in Quantum Annealers. In *LNCS*, Vol. 12296 LNCS. 112–129. doi:10.1007/978-3-030-58942-4_8 arXiv:1912.08314
- [2] Jun Cai, Wg Macready, and Aidan Roy. 2014. A practical heuristic for finding graph minors. *CoRR* abs/1406.2 (2014), 1–16. arXiv:arXiv:1406.2741v1 <http://arxiv.org/abs/1406.2741>
- [3] Elisabeth Lobe and Annette Lutz. 2024. Minor embedding in broken chimera and derived graphs is NP-complete. *Theoretical Computer Science* 989 (2024), 1–23. doi:10.1016/j.tcs.2023.114369
- [4] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2018. Neo: A Learned query optimizer. *PVLDB* 12, 11 (2018), 1705–1718. doi:10.14778/3342263.3342644 arXiv:1904.03711
- [5] Manuel Schönberger. 2022. Applicability of Quantum Computing on Database Query Optimization. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 2512–2514. doi:10.1145/3514221.3520257
- [6] PG G Selinger, MM M Astrahan, D D Chamberlin, R A Lorie, and T G Price. 1979. Access path selection in a relational database management system. In *SIGMOD*. 23–34. <http://dl.acm.org/citation.cfm?id=582095.582099>